Vector based amplitude detection for a First Order Ambisonics context: Poetic inquiries through machine listening

Vicente Yáñez Fuenzalida

Supervisor: Luc Döbereiner

Sound Studies and Sonic Arts, Universität der Künste Berlin

The project is the continuation of a research based on spatial sound, particularly Ambisonics, and the use of neural networks (NNs). As a stepping stone, the project *Inside the Noise*[1], finished in February of 2025, served as a disciplinary springboard. The creation of this piece unraveled unexpected musical interactions, and which in turn unfolded the use of Ambisonics with NNs in a creative and fertile way. Particularly — almost surprising — the project only occupied the frequency spectrum as data for analysis and training. Precisely, the analysis of the Ambisonic recordings, and the later classification of them, was carried out using Mel Frequency Cepstrum Coefficients (MFCCs), and performed only on the W channel of the Ambisonics stream. This operation willingly discarded spatial information encoded on the X, Y and Z channels—and beyond.[2] Nevertheless, the recordings were kept integral, spatially rich (most of them where 3rd and 5th order Ambisonics), as the playback system only used the label delivered by the classifier network and not the parsed stream. Once the training on the recordings was done, the classifier was then triggered using a spatial FM synthesis patch (the classifier would evaluate only the W channel of this new stream). The apparent mismatch of sources made the model predict and trigger the most uncanny configurations — and as expected —there was no spatial correlation between any of the recordings or the synthesis. Furthermore, the playback device was crafted in order for the recordings to overlap among themselves, and at the same time, with the feed of Fm synthesis that triggered the model. In addition to this, the playback system, a 3rd order Ambisonics dome (OTTOsonics), delivered a high resolution and clear output, so any relationships between sound materials — or lack thereof — was very transparent.

---

[1]Inside the Noise was a project developed during the Altrememorie Residency 25' made possible by LeSerre(Udine) and PiNA(Koper). The project was installed on a XII century church in central Koper (Slovenia). Documentation at https://vicenteyanez.com/inside-the-noise

[2] The reason behind this is may not be so exciting, as it was basically a technical limitation of processing power. The decision was made on the account of time within the residency and experimentation with uncontrollable variables. In this case, spatial information.

Whilst this artistic research gave a huge insight into the potential of spatial sound and NNs working together, there was a key absent element in the equation, which was actual spatial analysis. In this account, multichannel analysis appeared as a first natural step towards building a NN that could process spatial information and be actually trained on it.

The seed idea for this new endeavor, was to achieve a similar behavior like in *Inside the Noise*, where recordings were triggered — very much like in concatenative synthesis — but using the spatial information embedded in the evaluation data. The project that I follow to describe gets one step closer to that goal.

The model I present here investigates a machine-learning approach to estimating spatial orientation — or Direction of Arrival (DOA) — from a First Order Ambisonic (FOA) stream for real-time use within a MaxMSP/Spat5 environment. A custom FOA dataset was created with homogeneous formatting and discrete spatial labels. Instead of training directly on high dimensional time–frequency representations, the method derives amplitude-based spatial descriptors from the Ambisonic B-format signal, including intensity-vector statistics and complementary energy measures computed over analysis windows. These results are stored in CSV files, and compared with the dataset during training.

The training of the model and the deployment was mostly implemented during my EMS Stockholm residency (Sweden), between the 1st and 21st of September 2025. While there, I held an Open Studio session/artist talk, where I explained the nature of the project, the background/past projects that led to the creation of the model, and a little demonstration using an Ambisonics microphone and live monitoring for feedback.
The second stage of this project was carried out during my residency at Notam Oslo (Norway), between the 1st and 11th of March 2026. There, I enabled a pipeline for artistic use of the predictions of the model, in a form of a "prediction feedback" system, which I will describe later on.

<u>Building the dataset</u>

The thought process on how and why to build the dataset myself was mostly inspired by curiosity. Before any recordings or gathering happened, I studied the TAU dataset[3]: structure/order, metadata, sound files and labeling. Thus, I envisioned a comprehensive structure for the dataset, and consequently, was able to lay down a strategy and a timeline. The method used was as clean and thorough as possible, but I didn't intend for precise angle and elevation measurement, since I expected discrete class prediction from the model. In other words, the parceling of the 3d space was made forehand, so that the information gathered could be tailored to a specific size

[3] Sharath Adavanne et al., 'TAU Spatial Sound Events 2019 - Ambisonic and Microphone Array, Evaluation Datasets', Zenodo, 20 May 2019, https://doi.org/10.5281/ZENODO.3066123.

and quality. Moreover, taking these previous steps would help decide what kind of materials needed to be recorded.
The training of the model was done using FM synthesis and Ambisonic field recordings. In particular, the FM engine built inside MaxMSP in conjunction with Spat5, would allow for a diverse — slightly generative — output, but of a fairly homogenous quality. On the other hand, the studio were the recordings were made, was equipped with enough acoustic treatment and sufficient speaker resolution for precise sound localization. For the recording process, an octophonic planar speaker layout was used and a Zoom HR-VR microphone.
The synthesis was programmed to output short sounds in a sequential circular trajectory, not matching the pre-established azimuth classes directly. These trajectories were recorded at a central position, using three different elevation positions for the microphone, fairly matching the pre-established elevation classes. And then, similar sequences where recorded using the built-in reverb from Spat5, aiming to capture diffuseness — or rather unclear directionality. In addition to the microphone captures, a dry output from the patch (after the Spat5 decoder) was recorded as well, as part of the dataset.

After all the files were sorted and indexed, the structure for a metadata — the CSV file — was built. Notably, this step was key to achieve an architecture for the model itself. To create discrete descriptors, an algorithm was necessary for evaluating the training data — as the model would in a later stage — and output results that established class belonging for each of the sound impulses recorded. Here, the idea of a pseudo-intensity vector detection appeared as the most plausible solution[4]. After investigating some of the references mentioned in this documentation, I arrived to certain concepts and formulas that could detect DOA, and at the same time, establish levels of diffuseness — therefore ideal for evaluation of the training data and discrete label formatting. I will discuss these further when I talk about the 12-vector architecture.

<u>Working pipeline</u>

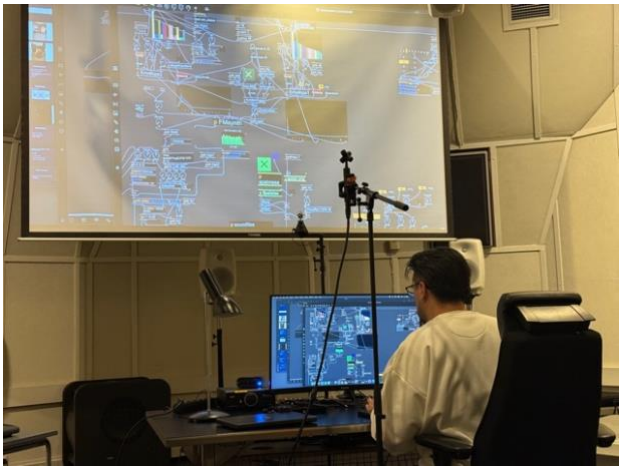
The first architecture probed for the model was a CRNN for sound event localization and detection (SELD)[5][6], but it was discarded midway due to the size of the dataset created, which was not enough for building a convolutional network. Moreover, the task itself could be achieved with a much simpler architecture and shorter training periods. Therefore, a deep feedforward neural network was trained to predict discrete

---

[4] Nils Poschadel et al., 'Investigations on Higher-Order Spherical Harmonic Input Features for Deep Learning-Based Multiple Speaker Detection and Localization', *EURASIP Journal on Audio, Speech, and Music Processing* 2025, no. 1 (2025): 8, https://doi.org/10.1186/s13636-025-00393-7.
[5] Francesca Ronchini et al., *SOUND EVENT LOCALIZATION AND DETECTION BASED ON CRNN USING DENSE RECTANGULAR FILTERS AND CHANNEL ROTATION DATA AUGMENTATION*, 2020.
[6] Poschadel et al., 'Investigations on Higher-Order Spherical Harmonic Input Features for Deep Learning-Based Multiple Speaker Detection and Localization'.

spatial attributes, specifically azimuth sectors (12), elevation classes (3), and overall diffusion (3), using cross-entropy objectives. Training applies class-balanced loss weighting across all three heads, and reinforces geometric structure in the azimuth prediction through an angular deviation penalty that discourages probability mass from concentrating on spatially distant sectors. After training, the model was exported (wrapped) for deployment and integrated into Max/MSP via the nn~ object[7].

Once in the Max/MSP environment, the vectorization process from the FOA stream needed to be replicated, in order to feed the model the same information it was trained on. Thus, a gen~ patch was created, which contains the same calculations, but now applied to a FOA stream in real-time. Consequently, the nn~ object outputs 18 values: 12 azimuth, 3 elevation and 3 diffuseness classes.
Every two seconds[8] — like the initial feature extraction window I will show later — the model will output these 18 values simultaneously. The patch allows the results to be stacked, listed and relayed into Spat5. I arbitrarily chose to use 16 sound objects [/sources], which are sequentially allocated in the Spat5 3D space, using the results predicted by the model. These positions are being constantly updated — creating a sort of generative spatialization engine, while being logged and compared in real-time, either with matrices for three dimensional vectors (calculating a dot product), or using circular variance calculations for 2D. These spherical statistics allow for real-time monitoring of the model's predictions in bulk, and consequently, the fertile usage of this data as source for something else.



Open studio session at EMS Stockholm, Sepember. 2025

[7] https://github.com/acids-ircam/nn_tilde/tree/master/scripting
[8] These windows can be tuned to be shorter, but as it goes against the training's nature, the errors get more frequent.

Feature Extraction and Spatial Encoding[9]

Spatial features are derived from the instantaneous intensity vector of the B-format signal, computed as the product of the omnidirectional channel W with each of the directional channels X, Y, and Z. The unit intensity vectors are then summarized over a 2-second window through: (a) Mean direction, (b) Standard deviation, (c) Mean resultant length [R], and (d) Diffuseness, alongside RMS-based energy ratios between the directional and omnidirectional channels.

Intensity-vector direction:

For each analysis window, intensity components are calculated between the channels:

- $I_x = W * X$
- $I_y = W * Y$
- $I_z = W * Z$

These are normalized – per sample – to obtain unit direction estimates:

$$\mathbf{u} = (u_x, u_y, u_z) = \frac{(I_x, I_y, I_z)}{\|(I_x, I_y, I_z)\|}$$

From the distribution of (u_x, u_y, u_z) across the window, the calculations extract:

a) Mean of direction components, representing the average directionality of the window (3): mean_x, mean_y, mean_z

b) Standard deviation of direction components, quantifying the temporal instability of each directional axis across the window (3): std_x, std_y, std_z

c) Resultant length, a scalar summary of directional concentration (1): R = sqrt(mean_x² + mean_y² + mean_z²)

d) Diffuseness, a scalar reflecting how concentrated — or not — the sound field is (1): $\psi = \text{clip}(1 - R) \in [0, 1]$

In parallel, energy descriptors (RMS) are computed directly from each channel (4):

- log_rmsW = log (rmsW + 1) — overall loudness
- Ratios relative to W: rX_ratio = rmsX/rmsW, rY_ratio, rZ_ratio — per-axis energy balance, independent of absolute loudness.

[9] Bastian Estay Zamorano et al., 'Sound Source Localization Using Hybrid Convolutional Recurrent Neural Networks in Undesirable Conditions', *Electronics* 14, no. 14 (2025): 2778, https://doi.org/10.3390/electronics14142778.

Since the directional features operate exclusively on unit vectors, magnitude information is entirely absent from the first group. The RMS descriptors restore this, and serve as a fallback in highly diffuse conditions where the intensity vectors become unreliable and R collapses toward zero.

These features form a stable, low-dimensional representation of FOA spatial behavior that can be learned efficiently by a dense network.

Loss Design and Training Strategy:

The model is trained as a multi-head classification network, predicting discrete spatial descriptors from the 12-D feature vector.

Output heads:

- Azimuth: discretized into 12 sectors (30° steps), trained with cross-entropy.
- Elevation class: trained with cross-entropy.
- ψ class (diffuseness): trained with cross-entropy, where ψ is converted into discrete classes using thresholds.[10]

Elevation and ψ are explicitly treated as imbalanced targets, because of the characteristics of the dataset. Training applies class-weighted cross-entropy to these heads, while preserving azimuth training performance. In addition to the azimuth cross-entropy, training adds an angular deviation penalty term computed from the azimuth softmax probabilities. This biases learning toward nearby azimuth sectors when the prediction is uncertain, rather than treating all wrong classes equally.

Training procedure:

Feature vectors are loaded from per-file CSVs and matched to their corresponding labels by timestamp. No input normalization is applied, which is a deliberate choice given that the RMS-based features already encode loudness in a log-compressed form, and the directional features are unit-normalized by construction during extraction.

The model is trained from scratch or resumed from an existing checkpoint, using AdamW with a learning rate scheduler that halves the rate when azimuth accuracy plateaus. Checkpointing is governed by a weighted improvement criterion that prioritizes azimuth accuracy while allowing beneficial trade-offs — a checkpoint is saved if azimuth improves, or if gains in elevation and diffuseness sufficiently outweigh a small azimuth regression (capped at 10%).

[10] Marc C. Green et al., 'Acoustic Scene Classification Using Higher-Order Ambisonic Features', *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, October 2019, 328–32, https://doi.org/10.1109/WASPAA.2019.8937282.

Deployment in Max/MSP (nn~ integration):

The trained model, al PyTorch script, was intended for real-time inference. For this reason, deployment on Max/MSP using IRCAM's nn~ was always a consideration. The wrapper available made by IRCAM suited the procedure as expected. Inside Max, a gen~ object replicates the exact same feature extraction pipeline used in the feature extraction process, so the network evaluates the same data structure it was trained on.

Moving forward, how to make use of the data outputted by the model, is nothing less than a substantial query. Without a concept for data instrumentalization — from an artistic point of view — the model will only serve the one purpose it was trained to do, which is, pointing towards a DOA. As mentioned earlier, the pipeline built after the deployed model (after the nn~ object), considers the listing and the vectorization/normalization of data, allowing for manageable temporal analysis of the behavior of the model[11]. This approach led to the idea of creating a feedback system between a live feed of FOA — using a FOA microphone — and real-time spatialization engine. To achieve this, and as a first approach to this challenge, I linked the initial FM patch (the one used for training) with the outcoming results from the predictions. The process works — very straightforwardly — by the means of gates and timers, which countdown to idle — or rather static — behavior of the model's output. The timers will trigger changes that stimulate movement and to avoid recursive behavior between the spatialization and the predictions. For example, they can trigger a reset on the Spat5 sources layout, or a preset configuration, using the pattr object, of the multichannel FM synthesis patch (which also works in a slightly generative way).

Open studio session at Notam, Oslo. March 26'

[11] See page 4 on the Working pipeline.

During my artistic residence at Notam (Norwegian Centre for Technology, Art and Music), I did trail-testing/data-observing exercises, which led to a fine-tuned system intended for long periods of performance — considering that the final goal was to put this "instrument" as a part of the audiovisual installation *Virtual Eschatologies* at UdK's SoundsAbout (Zwitschermaschine) gallery on March 20th, 2026.

```python
AZ_LOSS_WEIGHT = 1  # multiply azimuth head loss for stronger emphasis
ANG_PEN_WEIGHT = 1.2  # weight for angular deviation penalty

# — network architecture settings (update only here)
HIDDEN_DIMS = [512, 512, 4096, 4096, 512]  # sizes for fc1→fc2→fc3→fc4→fc5
DROPOUT = 0.01          # dropout probability

# ── model ──────────────────────────────────────────────────────────
class ThreeHiddenMLP(nn.Module):
    def __init__(self, in_dim=12, hidden_dims=HIDDEN_DIMS, dropout=DROPOUT):
        super().__init__()
        # unpack hidden layer sizes for five layers
        h1, h2, h3, h4, h5 = hidden_dims
        self.fc1 = nn.Linear(in_dim,  h1)
        self.fc2 = nn.Linear(h1,      h2)
        self.fc3 = nn.Linear(h2,      h3)
        self.fc4 = nn.Linear(h3,      h4)
        self.fc5 = nn.Linear(h4,      h5)
        self.dropout = nn.Dropout(dropout)
        # heads on final layer output
        self.az  = nn.Linear(h5, 12)
        self.el  = nn.Linear(h5, 3)
        self.psi = nn.Linear(h5, 3)

    def forward(self, x):
        x = F.relu(self.fc1(x)); x = self.dropout(x)
        x = F.relu(self.fc2(x)); x = self.dropout(x)
        x = F.relu(self.fc3(x)); x = self.dropout(x)
        x = F.relu(self.fc4(x)); x = self.dropout(x)
        x = F.relu(self.fc5(x)); x = self.dropout(x)
        return {'az': self.az(x), 'el': self.el(x), 'psi': self.psi(x)}
```

```python
# ── dataset ────────────────────────────────────────────────────────
class CSVDataset(Dataset):
    def __init__(self, index_path: Path):
        """Load features from consolidated index CSV"""
        df = pd.read_csv(index_path)
        self.X = []
        self.y_az = []
        self.y_el = []
        self.y_psi = []

        for r in df.itertuples():
            # load grouped per-wav CSV for this snippet
            df_feat = pd.read_csv(r.feature_csv)
            # find the row matching time_s
            match = df_feat[np.isclose(df_feat['time_s'], r.time_s)]
            if match.shape[0] != 1:
                raise ValueError(
                    f"Feature file {r.feature_csv} time_s={r.time_s} matches {match.shape[0]} rows"
                )
            # extract the 12 feature columns
            feats = match.iloc[0, :12].to_numpy(dtype=np.float32)
            self.X.append(feats)
            # labels
            # Fix azimuth: convert [-180,180] to [0,360] then to class
            az_deg_fixed = (r.azimuth_deg + 360) % 360
            self.y_az.append(int(az_deg_fixed // 30))
            self.y_el.append(int(r.elev_class))
            # Use the balanced psi_cls from index instead of computing from psi_overall
            self.y_psi.append(int(r.psi_cls))

        # just stack features; no normalization
        self.X = np.stack(self.X, 0)
        self.y_az = np.array(self.y_az)
        self.y_el = np.array(self.y_el)
        self.y_psi = np.array(self.y_psi)

        print(f"Loaded {len(self.X)} samples")
        print(f"Azimuth class distribution: {np.bincount(self.y_az)}")
        print(f"Elevation class distribution: {np.bincount(self.y_el)}")
        print(f"Psi class distribution: {np.bincount(self.y_psi)}")

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return (torch.from_numpy(self.X[idx]), self.y_az[idx], self.y_el[idx], self.y_psi[idx])
```

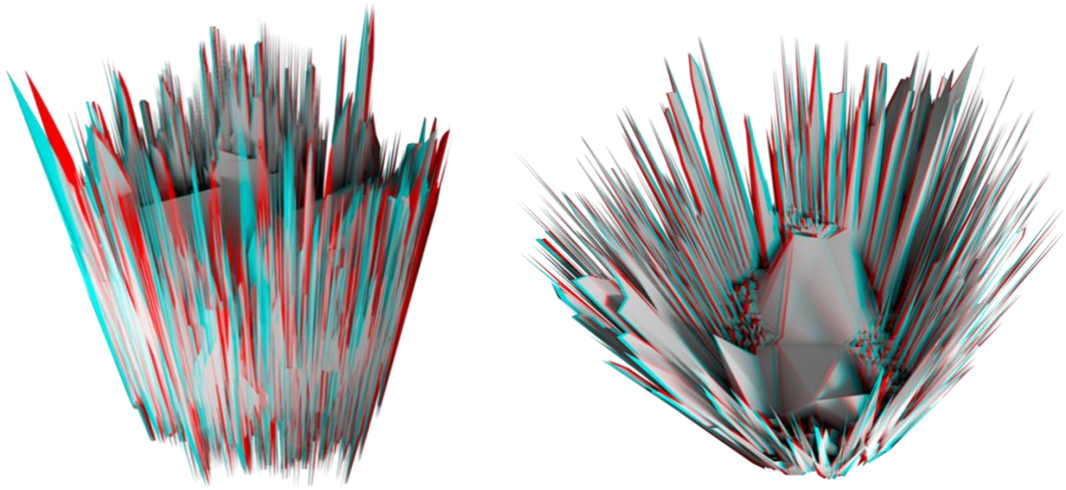
Pytorch script structure of the model

Poetic context:

This work initially began as a technical inquiry. My goal was to build a machine-listening approach to spatial sound; one that could propel soundscape composition and "musical" discourse by the means of procedural programming. Undoubtedly, I could reach for a poetic context that fits this research into an AI frame of reference. However, in parallel (for the last year or so), I have been creating virtual reality films, using different processes of translation of sound; as well as investigating the refraction of (virtual) light with Path Tracing. The universe portrayed in these films is populated with sculptures, which arose from encoding sound into geometries. But more importantly, these pieces are a poetic depiction of architectures, super-structures, that hold a double binding: a fictional reality, that shape-shifts constantly beyond human grasp or affection; and a self-sustaining/updating manifold of reflections of the real. The films (as a whole) are called *Inferno*, making a link with the religious construct the Conquistadores used as a measure of creed domination over Native Americans, while attempting to convert them to Christianity. On the other hand, the impossible — but apparently stable — structures that appear on the films depict a construct, a site that is neither animate or inanimate, not dead or undead: it transforms while being watched, it adapts to the human gaze (camera

movement in this case), as some kind of mirage[12]. The jittered reflections in this environment are the result of calculations of the refractions of light done by the Path Tracing, that even through the most uncanny shapes and materials, still pretend to emulate light's ubiquity, overarching the idea of an anticipation of the real[13].

Taking a couple of steps back, before the populated environment was finished, a byproduct arose in the midst of the research — like a key to open the underworld itself. In the attempt of creating the ideal architectures that would claim a site in this mirror world — and also following a thread about transduction — I put together a Python script that could output a 3D graphic based on a stereo sound file. Very straightforwardly, the code grabs amplitude and frequency, and lays the information over a discrete time, generating dimensionality by mapping these to height and distance to the central axis. The files used to output this particular shapes and architectures were recordings from places humanity has deemed sacred throughout history[14]. The procedure came to be, as part of an inquiry around sound and printed media. My thesis reached for metaphors and relations between the immaterial/metaphysic nature of the sacred and a possible transduction through sound. I used a version of this shapes in anaglyph form, as part of a zine publication carried by Alejandra Cárdenas[15].



Shapes created from stereo field recordings. They populate the Unreal Engine environment in the ***Inferno*** *films.*

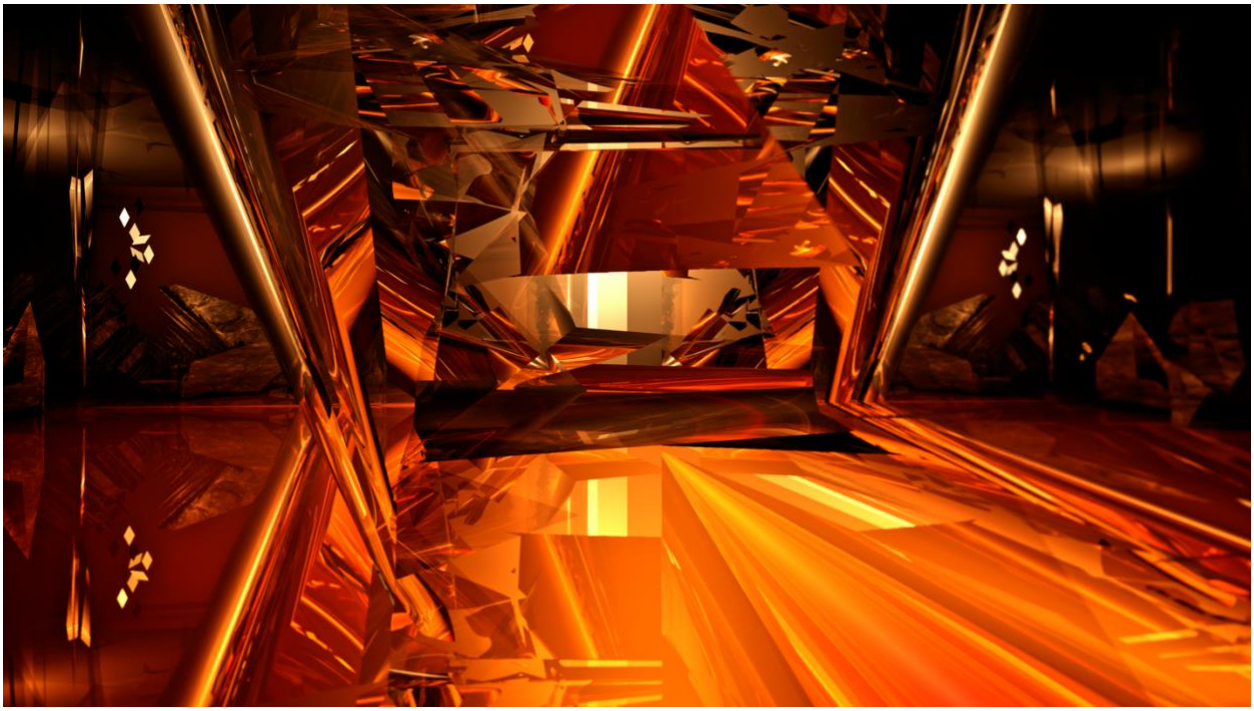
[12] Mirages — mirrors in particular – became extremely popular in the courts of the 17th and 18th century in Europe. As a display of power and luxury, but also like an allegoric turn towards anthropocentrism.

[13] See Mark Fisher's quote of Baudrillard's *Simulation and Simulacra*. Mark Fisher and Adam Jones, *Flatline Constructs: Gothic Materialism and Cybernetic Theory-Fiction*, Zer0 Classics (Zero Books, 2025), 41.

[14] The recordings are freely accessible at https://citiesandmemory.com/

[15] https://contingentsounds.com

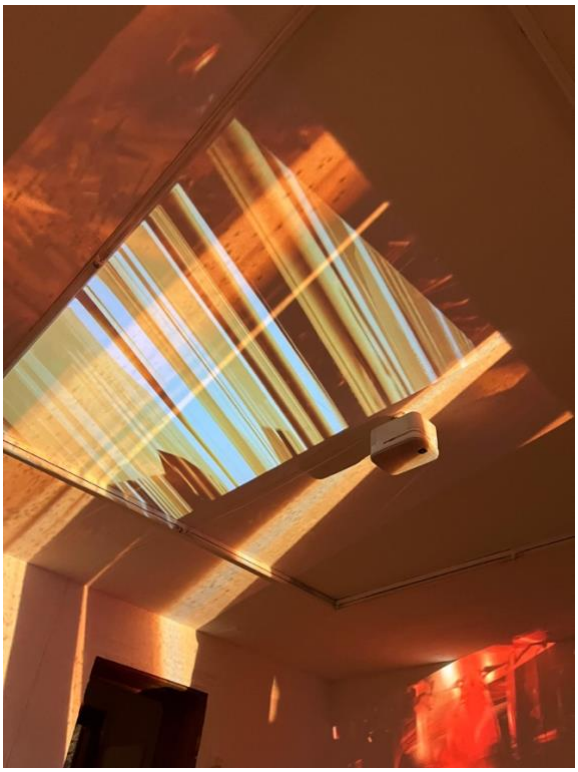
Adjacent to this work, the *machine-listening* ambisonics project reclaims a coherent spot at the heart of this *Inferno*, expanding the virtual loop into real-time acoustics, therefore actualizing the architectonic concept — as well as its reflecting/refracting nature — through sound and speculation.

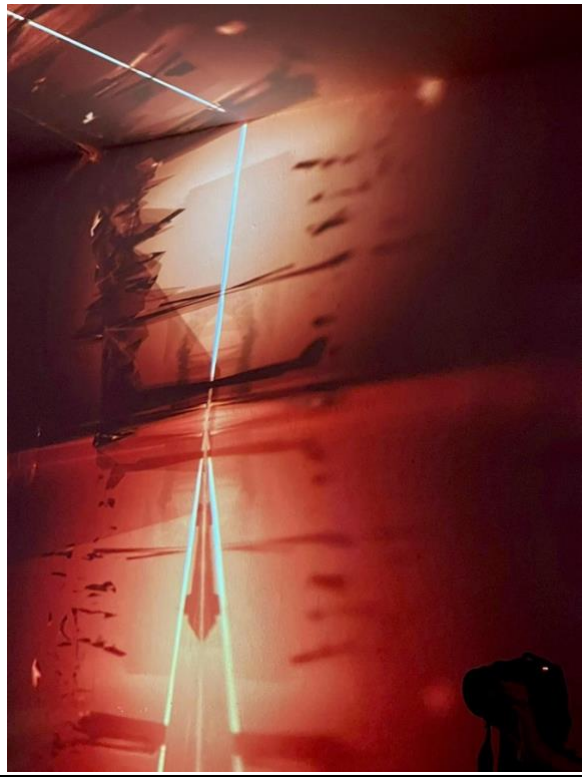


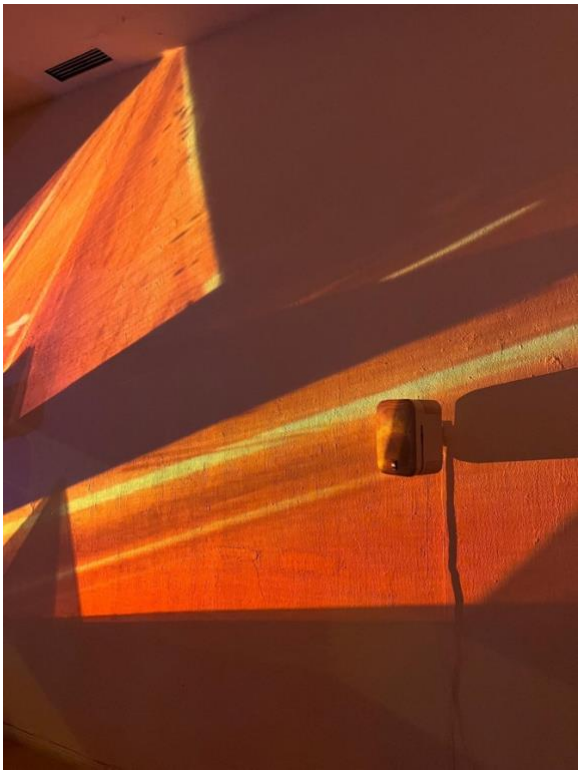
Audiovisual installation: *Virtual Eschatologies*

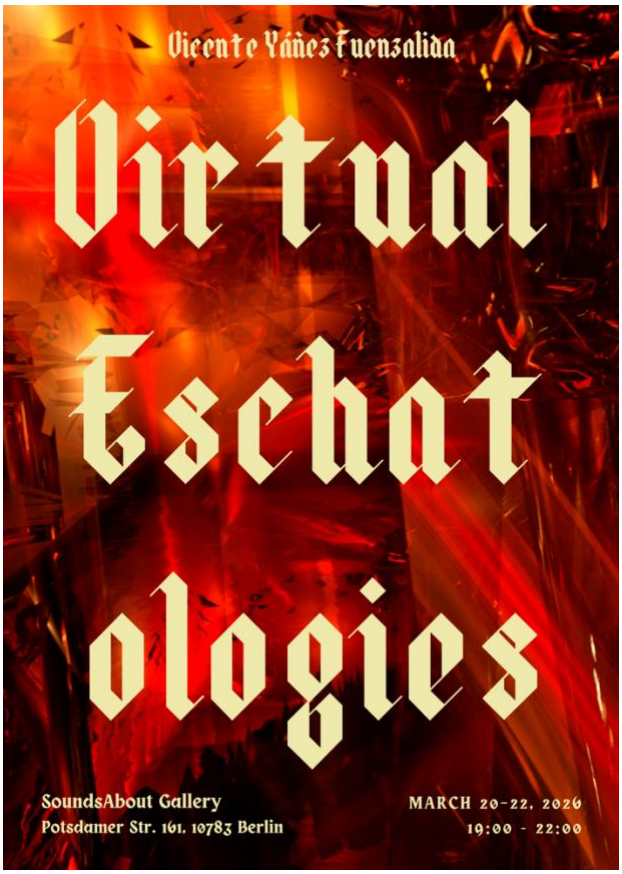
As a result, these concepts were put together in the form of an audiovisual installation at Zwitschermaschine Berlin. The machine listening device was deployed alongside the films, and a sculptural device born from the rendered shapes of the sacred — called *Temples*. The films where projected onto the walls of the gallery, and a FOA array was built in such a way the projections would not be severely interrupted by them. At the center of the space, a ZOOM VR-HR microphone, running the "feedback" loop between the model and the synthesis. Lastly, on a white plinth, a sculpture, a plexiglass polygon with water and with 3D printed versions of the shapes — the shapes we see on screen — that are, as explained before, part of this recursive process that concerns itself with the immanent, the sacred, and consequently as an alternative representation of eschatology.

This installation was running between the 20th and the 22nd of March 2026, during the Sound Studies and Sonic Arts' SoundsAbout winter season.









Inferno (Virtual Eschatologies)
Videos: Fixed media Unreal Engine 5
Sound: Realtime DOA Ambisonics feedback system

Temples (Virtual Eschatologies)
3D printed stereophonic heatmaps

Virtual Eschatologies takes as its point of departure the Jesuit imaginaries of hell that accompanied the "spiritual conquest" of the Americas in the 16th century; a colonial eschatology exported as universal truth, deployed to overwrite existing cosmologies. Here, that historical inferno is reconfigured as a virtual dimension, governed by its own architectural laws and built on reflections of the actual world – yet constantly disguising its origins through reflection, refraction and jitter.

The piece also brushes against the abstractions of techno-capitalist infrastructures – those self-sustaining circuits, floating between the immaterial and the brutally material, arise as metaphoric encounters within and among the erratic refractions, monumental architectures, and the tracings of light. Here, their persistence is not displayed as a data feed but as a question of duration; a sound device is deployed as a construct, an algorithm, that becomes present through sound detection and directionality. The listening device encodes positionality as an agent that affects a stream of synthesis, which is forwarded back into the space. A repeated strum marks time – a perpetual pulse that feeds on the actual while pushing toward self-transformation. Inferno stages the double bind: a space where colonial eschatologies and techno-capital architectures fold into one another, producing a continuously shifting elsewhere that is inseparable from the present.

The accompanying sculptural piece, Temples, appears as a byproduct of the process. Its shapes – also visible on screen–emerge from a dialogue between field recordings of sites humanity has deemed "sacred" across the world, code, and 3D modelling.

Universität der Künste Berlin

SOUNDS . ABOUT

Afterword:

Retrospectively, the installation wrapped this seven-month-long project, in a way that not only reframed artistically the FOA model, but also encapsulated my research with virtual environments and gothic materialist thinking[16]. As a strategy, I believe all the technical developments involved in this project should be developed in their time and manner — not necessarily as a joint venture. Particularly, the next logical step with the Ambisonics model would be to include frequency analysis, and cross-train the model with amplitude detection. Furthermore, I'm interested on the performance capabilities of a model that could treat frequency and spatiality as a feed, and consequently, propel feedback processes for live situations. On the other hand, the research regarding virtual environments pretends to link the deployment of an application for VR and sound detection on a further level.

[16] Fisher and Jones, *Flatline Constructs*.

Links to documentation:

Virtual Eschatologies:

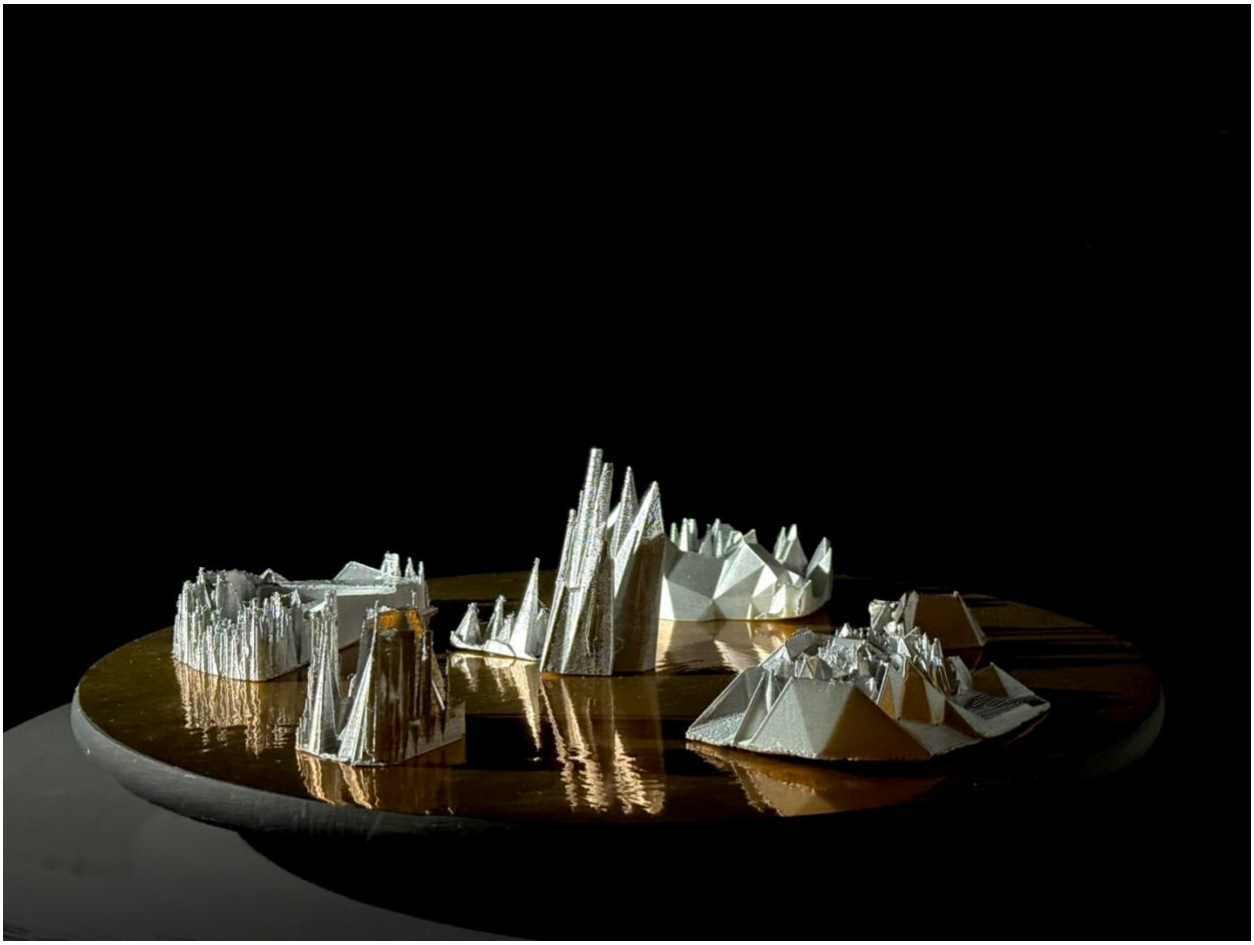
https://vimeo.com/1176879599

Inferno (excerpt):

https://vimeo.com/1158011666

Vernissage:

https://www.instagram.com/p/DWJVzetigw9



References:

Adavanne, Sharath, Archontis Politis, and Tuomas Virtanen. 'TAU Spatial Sound Events 2019 - Ambisonic and Microphone Array, Evaluation Datasets'. Zenodo, 20 May 2019. https://doi.org/10.5281/ZENODO.3066123.

Estay Zamorano, Bastian, Ali Dehghan Firoozabadi, Alessio Brutti, et al. 'Sound Source Localization Using Hybrid Convolutional Recurrent Neural Networks in Undesirable Conditions'. *Electronics* 14, no. 14 (2025): 2778. https://doi.org/10.3390/electronics14142778.

Fisher, Mark, and Adam Jones. *Flatline Constructs: Gothic Materialism and Cybernetic Theory-Fiction*. Zer0 Classics. Zero Books, 2025.

Green, Marc C., Sharath Adavanne, Damian Murphy, and Tuomas Virtanen. 'Acoustic Scene Classification Using Higher-Order Ambisonic Features'. *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, October 2019, 328–32. https://doi.org/10.1109/WASPAA.2019.8937282.

Poschadel, Nils, Stephan Preihs, and Jürgen Peissig. 'Investigations on Higher-Order Spherical Harmonic Input Features for Deep Learning-Based Multiple Speaker Detection and Localization'. *EURASIP Journal on Audio, Speech, and Music Processing* 2025, no. 1 (2025): 7. https://doi.org/10.1186/s13636-025-00393-7.

Ronchini, Francesca, Daniel Arteaga, and Andres Perez-Lopez. *SOUND EVENT LOCALIZATION AND DETECTION BASED ON CRNN USING DENSE RECTANGULAR FILTERS AND CHANNEL ROTATION DATA AUGMENTATION*. 2020.